

한국어 통사 자동 분석기 (Streamlit + Kiwi + Gemini)

Streamlit 기반 웹 앱으로 한국어 문장의 통사 구조(안긴문장 종류, 문장 성분, 문장 종류)를 자동 분석한다. 핵심 분석은 Kiwi 형태소 분석기 위에 구축한 규칙 기반 엔진으로 수행하고, 보유한 Gemini API를 옵션으로 연동해 모호한 사례 검증과 자연어 설명을 보조한다.

1. 목표 및 분석 범위

학교 문법 기준의 통사 분석을 자동화한다. 출력은 다음 세 층으로 구성된다.

- 형태소 층:** Kiwi 결과를 어절별로 정리한 표 (형태소·품사·기능 표시)
- 문장 성분 층:** 주어·서술어·목적어·보어·관형어·부사어·독립어를 색상으로 강조
- 절 구조 층:** 안긴문장(명사절·관형절·부사절·서술절·인용절)과 이어진 문장(대등/종속)을 트리로 시각화하고, 전체 문장 종류(홀문장/겹문장)를 요약

분석 대상은 학교 문법에서 다루는 표준 한국어 단문/복문이며, 구어체·신조어는 best-effort.

2. 기술 스택과 디렉터리 구조

- 언어: Python 3.10+
- UI: Streamlit (단일 앱, 브라우저 표시)
- 형태소 분석: kiwipiepy (Kiwi)
- 트리 시각화: streamlit.graphviz_chart 로 DOT 문자열 렌더 (시스템 Graphviz 설치 불필요)
- LLM 보조: google-generativeai (Gemini), .env 에 GEMINI_API_KEY 가 있을 때만 활성화
- 테스트: pytest

```
f:\통사자동분석기\  
├─ app.py                                # Streamlit 진입점 (UI/렌더링)  
├─ analyzer/  
│  ├── __init__.py  
│  ├── models.py                        # dataclass: Morph, Eojeol, Clause, Component, Analysis  
│  ├── tokenizer.py                    # Kiwi 래퍼: 형태소 분석 + 어절 그루핑  
│  ├── clause_detector.py              # 안긴문장/이어진문장 식별  
│  ├── component_analyzer.py          # 주어·서술어·목적어·보어·관형어·부사어·독립어 식별  
│  ├── sentence_classifier.py          # 홀/겹, 안은문장/이어진문장 분류  
│  ├── pipeline.py                    # 위 모듈을 연결하는 단일 진입 함수 analyze(text)  
│  ├── visualizer.py                  # graphviz DOT 생성, HTML 색상 강조 생성  
│  └─ llm_assistant.py                 # Gemini 호출 래퍼 (옵션)  
├─ examples/sample_sentences.txt        # UI 예제 드롭다운 데이터  
├─ tests/  
│  ├── test_clause_detector.py  
│  ├── test_component_analyzer.py  
│  └─ test_pipeline.py  
├─ requirements.txt  
├─ .env.example  
└─ README.md
```

3. 데이터 모델 (analyzer/models.py)

dataclass 기반으로 정의한다. 핵심 필드만 명시:

- Morph : surface , lemma , tag (Kiwi POS), start , end
- Eojeol : text , morphs: list[Morph] , index
- Component : kind (SUBJ|PRED|OBJ|COMPL|ADN|ADV|INDEP), eojeol_indices , note
- Clause : kind (MAIN|NOUN|ADN|ADV|PRED|QUOT|COORD|SUBORD), span (어절 범위), head_eojeol , marker_morph , children: list[Clause] , components: list[Component]
- Analysis : text , eojeols , root: Clause , sentence_type (홀문장/겹문장 + 세부), notes: list[str]

4. 분석 파이프라인 (analyzer/pipeline.py)

```
analyze(text) → Analysis
1) tokenize(text)           # Kiwi → Eojeol 리스트
2) detect_clauses(eojeols)  # 절 트리 구축 (root: MAIN)
3) analyze_components(root) # 각 절 내부에 성분 부착
4) classify_sentence(root)   # 문장 종류 결정
```

각 모듈은 순수 함수로 작성하고 Analysis 만 반환해 UI/LLM에서 동일하게 사용한다.

4.1 Kiwi 토크나이저 (tokenizer.py)

- Kiwi() 인스턴스를 모듈 전역 1회 생성 (Streamlit @st.cache_resource).
- kiwi.tokenize(text) 결과를 어절(공백) 경계 기준으로 묶어 Eojeol 리스트 반환.
- 종결어미 EF 가 나오면 문장 경계 후보로 표시.

4.2 안긴문장/이어진문장 식별 (clause_detector.py)

Kiwi POS 태그 기반 결정 규칙:

절 종류	트리거	근거 형태소 예
명사절	(a) ETN 명사형 전성어미 직접 / (b) ETM + 의존명사 것/바/줄/데 직후 격조사 결합	-(으)ㅁ , -기 , -는 것
관형절	ETM 뒤에 체언(NN*/NP/NR)이 오고, 그 체언이 의존명사가 아니거나 동격 명사구가 아님	-(으)ㄴ , -는 , -(으)ㄹ , -던
부사절	(a) EC 중 부사화 어미 화이트리스트, (b) 부사 파생접미사 -이(없이/같이)	-게 , -도록 , -듯이 , -이 , -아서/어서 (부사적 용법)
인용절	직접: 따옴표 + JKQ (라고/하고) / 간접: EC -고 + 발화·사유 동사(말하다/생각하다/믿다 등)	-고 , -라고
서술절	형식적 표지 없음. 한 절 안에 jks 주어가 둘 이상이고 두 번째 주어가 별도 서술어를 갖는 경우 (이중 주어 구문)	(없음)
이어진 (대등)	EC 중 대등 연결어미 화이트리스트	-고 , -며 , -(으)나 , -거나 , -든지
이어진 (종속)	EC 중 종속 연결어미 화이트리스트	-(으)면 , -(으)니 , -아서/어서 , -(으)므로 , -(으)려고 , -(으)러

알고리즘:

1. 어절 리스트를 좌→우 스캔하며 위 트리거가 있는 어절을 절 경계로 표시.
2. MAIN 노드를 루트로 두고, 트리거가 발견될 때마다 그 어절을 head로 하는 자식 clause 노드를 생성, 좌측 어절들을 그 절의 span으로 귀속.

- 인용절은 따옴표 범위를 우선 사용.
- 서술절은 다른 절 분류가 끝난 뒤 마지막에 같은 절 내 이중 주어 검사로만 식별.
- 동일 어절이 여러 트리거를 가질 때 우선순위: 인용 > 명사 > 관형 > 부사 > 이어진.

4.3 문장 성분 분석 (component_analyzer.py)

각 clause 내부에서:

- 격조사 기반 1차:
 - JKS → 주어, JKO → 목적어, JKC → 보어, JKB → 부사어
 - JKG → 관형격(앞 어절은 관형어), JKV → 호격(독립어)
- 보조사(JX)로 표지된 어절: 문맥상 주어/목적어 추정 (서술어와의 관계로 1차 추론, 모호하면 note 에 기록)
- 서술어: 동사/형용사 어간 + EF (또는 절의 head 어미) → 그 어절을 PRED
- 관형어: 관형사(MM), ETM 로 끝나는 용언 어절 (관형절의 head이기도 함)
- 부사어: 부사(MAG / MAJ), JKB 결합 어절, 부사절 head
- 독립어: 감탄사(IC), 호격
- 서술절 사례에서는 두 번째 주어와 그 서술어가 내포 절을 이루므로, clause_detector 가 이미 PRED 자식 노드를 만든 뒤 성분 분석은 각 노드 단위로 분리 수행

4.4 문장 종류 분류 (sentence_classifier.py)

- 자식 절 수 0 → 홀문장
- 자식 절 ≥ 1 → 겹문장. 자식 종류로 세부 라벨 부여:
 - 명사절/관형절/부사절/서술절/인용절을 가진 경우 → "안은 문장 (xx절을 안은)"
 - COORD / SUBORD 를 가진 경우 → "이어진 문장 (대등/종속)"
 - 둘 다면 두 라벨 결합

5. UI 구성 (app.py)

Streamlit 위젯으로 단일 페이지를 구성한다.

- 사이드바
 - 예문 드롭다운 (examples/sample_sentences.txt 에서 로딩)
 - 「AI 보조 설명 사용」 토글 (GEMINI_API_KEY 가 있을 때만 활성화)
 - 「색상 범례」
- 본문
 - 문장 입력 텍스트박스 + 「분석」 버튼
 - 결과 탭 4개:
 - 요약:** 문장 종류 카드, 안긴 절 목록 칩
 - 문장 성분:** 어절별 색상 강조 HTML (st.markdown(unsafe_allow_html=True))
 - 절 구조 트리:** st.graphviz_chart 로 DOT 렌더 — 노드 라벨에 절 종류와 head 어미, 자식 관계를 표시
 - 형태소 표:** st.dataframe 으로 토큰·POS·기능
 - AI 토글이 켜져 있으면 결과 하단에 「AI 보조 설명」 expander 추가

6. Gemini 보조 모듈 (analyzer/llm_assistant.py)

- python-dotenv 로 .env 로드, GEMINI_API_KEY 또는 GOOGLE_API_KEY 둘 다 허용.
- google.generativeai.configure(api_key=...) → GenerativeModel("gemini-2.5-flash") .
- 함수 explain(analysis: Analysis) → str :

- 입력: 원문 + 규칙 기반 분석 결과(JSON 직렬화)
 - 프롬프트: "다음은 학교 문법에 따라 자동 분석된 한국어 문장이다. 결과의 정확성을 검토하고 잘못된 부분을 한국어로 지적하며, 학습자에게 도움이 되도록 각 안긴 절 성분을 자연스럽게 설명하라. 분석 결과를 멋대로 다시 만들지 말고 검토와 부연만 하라."
 - 출력: 마크다운 텍스트
- API 호출 실패 시 사용자에게 메시지 노출하고 규칙 기반 결과는 그대로 유지.
 - 키 미설정 시 토글이 비활성화되며 "Gemini API 키 미설정" 안내.

7. 의존성과 환경 설정

requirements.txt (버전은 설치 시 최신으로 핀):

- streamlit
- kiwipiepy
- google-generativeai
- python-dotenv
- pandas (데이터프레임 표시용)
- pytest (개발용)

.env.example :

GEMINI_API_KEY=

README.md 에 다음을 명시:

- 가상환경 생성 (python -m venv .venv , PowerShell 활성화 명령)
- pip install -r requirements.txt
- .env 작성
- 실행: streamlit run app.py

8. 검증 (tests/)

각 절 종류 표준 예문을 fixture로 두고 pipeline.analyze(text) 결과의 절 트리 구조와 라벨을 단위 테스트한다.

케이스	예문	기대 결과
명사절 (-(으)ㄴ)	그가 범인임이 밝혀졌다.	자식 1개 = 명사절
명사절 (-기)	나는 그가 떠나기를 바랐다.	명사절(목적어 위치)
명사절 (-는 것)	비가 오는 것이 보인다.	명사절(주어 위치)
관형절 (관계)	내가 어제 만난 사람은 친절하다.	관형절 + 성분 생략 표시
관형절 (동격)	그가 떠났다는 사실은 슬프다.	관형절(동격)
부사절 (-이)	비가 소리도 없이 내린다.	부사절
부사절 (-게)	꽃이 아름답게 피었다.	부사절
서술절	코끼리는 코가 길다.	서술절 (이중 주어)
인용절 (직접)	그는 "내일 가겠다"라고 말했다.	직접 인용절
인용절 (간접)	그가 온다고 했다.	간접 인용절
이어진(대등)	비가 오고 바람이 분다.	대등 연결
이어진(종속)	비가 와서 길이 미끄럽다.	종속 연결

수동 검증 절차:

- 1. pytest -q 통과
 - 2. streamlit run app.py 실행 후 위 12개 예문을 사이드바에서 차례로 선택해 4개 탭 결과 점검
 - 3. AI 토글을 켜 상태에서 동일 예문 1~2개로 Gemini 응답 표시·실패 메시지 모두 확인 (키 제거 후 비활성 표시 확인)
- ### 9. 한계와 보강 정책
- 규칙 기반 분석은 형태소 표지에 의존하므로 다음의 모호한 사례는 Analysis.notes 에 경고를 남긴다.
 - -기 vs -기 때문에 같은 굳어진 표현
 - 보조사 은/는 이 주어와 주제를 모두 표시할 수 있는 경우
 - 종속/대등 연결어미가 동일 형태(-고)일 때
 - AI 토글은 위 경고가 있는 결과에 대해 자동으로 추가 설명을 우선 노출한다.
 - 향후 보강은 Kiwi 사용자 사전 추가, 화이트리스트 어미 확장, 테스트 케이스 추가로만 처리하고 모델 학습은 도입하지 않는다 (해석 가능성 우선).

10. 작업 항목 (todos)

- [] scaffold — 프로젝트 스캐폴드 생성: 디렉터리 구조, requirements.txt , .env.example , README.md , examples/sample_sentences.txt
- [] models — analyzer/models.py 에 Morph / Eojeol / Component / Clause / Analysis dataclass 정의
- [] tokenizer — analyzer/tokenizer.py 에서 Kiwi 인스턴스 캐시 + 어절 그루핑 + 종결어미 경계 표시 구현
- [] clause_detector — analyzer/clause_detector.py 에서 명사절·관형절·부사절·인용절·서술절·대등/종속 연결 식별 규칙 구현 (트리 구축 포함)
- [] component_analyzer — analyzer/component_analyzer.py 에서 절 단위 주어·서술어·목적어·보어·관형어·부사어·독립어 식별 구현
- [] sentence_classifier — analyzer/sentence_classifier.py 에서 홀/겹문장 및 안은문장·이어진문장 세부 라벨링 구현
- [] pipeline — analyzer/pipeline.py 에 analyze(text) 통합 함수 작성
- [] visualizer — analyzer/visualizer.py 에서 절 트리 DOT 문자열과 어절 색상 강조 HTML 생성

- [] **llm** — analyzer/llm_assistant.py 에서 Gemini 호출 래퍼와 키 부재 시 비활성 처리 구현
- [] **ui** — app.py 에 사이드바·입력·4개 결과 탭(요약/성분/트리/형태소)과 AI 토글을 가진 Streamlit UI 작성
- [] **tests** — tests/ 아래에 절 종류별 표준 예문 단위 테스트 작성 후 pytest 통과 확인
- [] **manual_qa** — streamlit run 으로 12개 예문 수동 검증 + Gemini 토글 동작 확인 + README 의 설치/실행 절차 점검